Momentum AI: Architecture & The Path to Autonomous App Control

> [!NOTE]
> **What is Momentum?**
> Momentum is a lightning-fast, highly conversational autonomous AI agent built natively for macOS. It currently features ultra-low latency speech-to-text (STT), human-like text-to-speech (TTS), and system-level context awareness. It acts as a foundation for a fully autonomous operating system companion.

The next major leap for Momentum is the **App Control & Automation Module**. This module will give the agent the ability to physically control native desktop applications, navigate complex UIs, and execute multi-step workflows.

To achieve maximum speed and reliability, this module will be built in three distinct phases, strictly avoiding slow OCR (Optical Character Recognition) techniques in favor of programmatic native hooks.

Phase 1: The Accessibility Inspector (The "HTML" of Native Apps)

If Momentum relies purely on Vision models to read the screen, it will be incredibly slow and prone to clicking the wrong things. Just like a web browser has a DOM tree (HTML), macOS native applications have an Accessibility Tree.

How it Works

We will build a high-speed **UI Debugger Module** written in Rust, leveraging Apple's native AXUIElement (Accessibility APIs) and CoreGraphics.

1. **Tree Extraction**: The instant you open an app (e.g., Spotify, Chrome, Xcode), Momentum will silently query the macOS Accessibility API to extract a complete hierarchy of the application's current state.
2. **Real-time Semantic Labeling**: Every UI element will be mapped into a lightweight JSON structure in milliseconds, bypassing vision models entirely:

```
{
  "app": "Spotify",
  "window_title": "Daily Mix 1",
  "elements": [
    {"id": 1, "role": "Button", "name": "Play", "bounds": {"x": 100, "y": 200}},
    {"id": 2, "role": "TextField", "name": "Search", "bounds": {"x": 50, "y": 20}}
  ]
}
```

3. **The Debugger HUD**: We will build a developer overlay tool that visually draws colored boxes around these elements in real-time on your screen. This will allow us to see exactly what Momentum's "blind" programmatic eyes are seeing, ensuring every button and function is labeled correctly.

Phase 2: The Fast-Action Planner (The Brain)

Once Momentum can programmatically read the UI tree of any application in under 50 milliseconds, it needs to know what to do with that information.

The Routing Architecture

We will introduce a blazing-fast local routing model that acts as the "Decision Engine".

- **State Injection**: The UI JSON tree is flattened into a highly compressed, token-efficient text representation and injected directly into the LLM's context.
- **Skill-Based Matching**: When the user says *"Play some jazz"*, the Planner evaluates the intent against:
    1. The available buttons on the screen.
    2. Momentum's predefined native "Skills" (e.g., AppleScript hooks, terminal commands).
- **Execution Engine**: Once the model decides to click the "Search" button and type "Jazz", the Rust backend will use CGEventCreateMouseEvent to fire physical mouse clicks and keyboard events directly at the (x, y) coordinates of the labeled button.

> [!TIP]
> **Why this matters**: Because we aren't waiting for a 7-Billion parameter Vision model to process a screenshot, Momentum will be able to navigate an app, click 5 buttons, and type a response in the time it takes a standard AI to even realize the screen changed. Fast as fuck.

Phase 3: The Ultra-Lightweight Vision Fallback

There are certain things the macOS Accessibility API cannot read. Custom-rendered game engines, complex canvas elements, images, and **videos** do not have semantic labels.

The 8GB Hardware Constraint

For these edge cases, Vision is required. To ensure this runs smoothly on standard 8GB RAM hardware without choking the system:

1. **Quantized Micro-Vision Models**: We will deploy an ultra-lightweight, heavily quantized (4-bit) vision model (such as *Moondream2* or a stripped-down *Qwen-VL*) that is small enough to stay persistently loaded in VRAM alongside the main conversational brain.
2. **Triggered Perception**: Vision will **never** run continuously. It will only be triggered when:
    - The Accessibility Inspector returns an opaque Canvas or WebArea with no semantic children.
    - The user explicitly requests visual analysis (e.g., *"What is happening in this video?"*).
3. **Video Processing**: By using a micro-vision model, Momentum can sample 1-2 frames per second of a video stream and analyze it in near real-time without melting the user's GPU.

Summary of the Architecture Flow

```
graph TD
    A[User Command] --> B[Momentum Audio Engine]
    B --> C[Fast-Whisper STT]
    C --> D{Is it a Visual Task?}

    D -- No --> E[Conversational Brain]

    D -- Yes --> F[Rust Accessibility API]
    F --> G[Extract Semantic UI Tree]

    G --> H{Are Elements Opaque?}

    H -- No --> I[Fast-Action Planner]
    I --> J[Execute Native Clicks / Typing]

    H -- Yes --> K[Trigger Micro-Vision Model]
    K --> I
```